Desafio Técnico — Sistema de Precificação e Gestão de Posição em DI1

1. Objetivo

Desenvolver uma aplicação web para precificar contratos futuros de DI1 e acompanhar a posição de uma carteira ao longo do tempo.

O projeto deve permitir que um usuário cadastre contratos, registre operações de compra e venda, controle movimentações de caixa e consulte como sua carteira estava em uma data específica.

Não é necessário ter conhecimento prévio sobre o mercado financeiro. Todas as regras de negócio necessárias para a implementação estão descritas neste documento.

2. Contexto do problema

Contratos futuros de DI1 são instrumentos financeiros cujo preço está relacionado à taxa de juros e ao prazo até o vencimento.

Neste desafio, construiremos uma versão simplificada de um sistema que permite:

- Calcular o preço de referência de um contrato a partir de uma taxa de juros, ou calcular a taxa a partir de um preço.
- Registrar operações de compra e venda de contratos de DI1.
- Registrar entradas e saídas de dinheiro da carteira.
- Consultar a posição da carteira em uma data específica.

A aplicação não precisa consultar preços reais de mercado. Os preços de referência serão informados pelo próprio usuário.

3. Precificação dos contratos

Cada contrato deve possuir, no mínimo, as seguintes informações:

- Identificador ou código do contrato(Nome identificador).
- Data de referência do cálculo.
- Data de vencimento.
- Taxa de juros anual ou preço unitário (PU).

O sistema deverá calcular o valor que estiver faltando, conforme os dados informados.

Fórmulas

O preço unitário (PU) é calculado por:

PU = 100.000 / (1 + taxa)^(dias corridos / 365)

A taxa de juros pode ser calculada inversamente:

taxa = (100.000 / PU)^(365 / dias corridos) - 1

Onde:

- PU: preço unitário teórico do contrato.
- taxa: taxa de juros anual em formato decimal. Por exemplo, 10% deve ser representado por 0,10.
- dias corridos: quantidade de dias corridos entre a data de referência e a data de vencimento.
- 365: convenção anual utilizada nesta versão simplificada.
- 100.000: valor de referência utilizado na fórmula.

Para este exercício, utilize dias corridos, sem necessidade de calendário de dias úteis ou feriados.

Exemplo

Considere um contrato com:

- Data de referência: 01/01/2027.
- Data de vencimento: 01/01/2028.
- Taxa de juros: 10% ao ano.

PU = 100.000 / (1 +0.1) ^ (365 / 365) = 100 / 1.1 = 90.909,0909091

Aplicando a fórmula, o sistema deverá calcular o PU correspondente. Se o usuário informar esse PU posteriormente, o sistema deverá ser capaz de recuperar a taxa utilizada, respeitando eventuais diferenças de arredondamento.

Importante: a fórmula é uma simplificação criada para o desafio. Não é necessário reproduzir todas as regras operacionais de precificação e liquidação utilizadas pela B3.

4. Registro de operações e movimentações financeiras

O sistema deverá permitir que o usuário registre dois tipos de eventos.

4.1. Operações com contratos

O usuário deverá conseguir registrar uma operação de compra ou venda contendo:

- Contrato e vencimento.
- Tipo de operação: compra ou venda.
- Quantidade de contratos.
- PU ou taxa do contrato.
- Data da operação.

As operações deverão ser armazenadas como registros individuais, preservando o histórico. A posição atual não deverá ser a única informação persistida, pois deverá ser possível reconstruir a carteira em datas anteriores.

4.2. Movimentações de caixa

O usuário deverá conseguir registrar entradas e saídas de dinheiro, contendo:

- Tipo de movimentação: entrada ou saída.
- Valor.
- Data da movimentação.
- Descrição opcional.

Por exemplo, o usuário poderá registrar uma entrada de R$ 400.000,00 realizada dois dias antes da data atual.

O saldo de caixa em uma determinada data será calculado pela soma das entradas menos as saídas registradas até aquela data.

5. Consulta histórica da carteira

A aplicação deverá disponibilizar uma tela na qual o usuário escolha uma data e consulte a situação da carteira naquele momento.

A consulta deverá apresentar, no mínimo:

- Posição líquida por contrato.
- Quantidade de contratos em cada posição.
- PU de referência utilizado para a avaliação.
- Resultado financeiro simplificado da posição.

Exemplo

Operação 1

- Data de referência: 01/01/2027.
- Entrada de caixa: 200.000

Operação 2

- Identificador: DI1F28
- Data de referência: 01/01/2027.
- Data de vencimento: 01/01/2028.
- Taxa de juros: 10% ao ano.
- Compra de 1 contrato
- Financeiro operação: -90.909,09

Visão da posição no dia 01/01/2027

- Financeiro total: 200.000
- Posições:

```
---   ATIVO  |    QTD    | PREÇO
---   BRL    | 109090.91 | 1
---   DI1F28 |     1     | 90.909,09
```

A consulta deverá considerar apenas as operações e movimentações ocorridas até a data selecionada, além dos preços de referência disponíveis para a avaliação.

7. Requisitos funcionais

A aplicação deverá permitir:

1. Cadastrar e consultar contratos: registrar contratos e suas informações de precificação.
2. Calcular preços e taxas: informar um dos valores e calcular o outro.
3. Registrar operações: incluir compras e vendas de contratos.
4. Registrar movimentações de caixa: incluir entradas e saídas de recursos.
5. Informar preços de referência: registrar o PU de referência de um contrato para uma data.
6. Consultar a carteira por data: reconstruir as posições e o saldo de caixa de acordo com os eventos registrados até a data selecionada.
7. Consultar o histórico: visualizar as operações e movimentações já registradas.

8. Requisitos técnicos

Obrigatórios

- Backend: Python com FastAPI.
- Frontend: React.
- Persistência: banco de dados relacional.
- Execução: aplicação executável em ambiente Docker.
- Documentação: instruções para instalação, configuração e execução.

A organização interna do código, a modelagem do banco, a estrutura das APIs e a estratégia de atualização da interface ficam a critério do candidato, desde que as regras de negócio sejam respeitadas.

Desejáveis

- Processamento assíncrono.
- Uso de mensageria para processamento de eventos ou cálculos.
- Testes automatizados de regras de negócio e APIs.
- Indicação visual do estado dos cálculos, como pendente, em processamento, concluído ou com erro.

- Boas práticas de organização, tipagem, validação e tratamento de erros.

O uso de processamento assíncrono e mensageria é especialmente desejável para candidatos de nível pleno ou superior. Não é necessário utilizar processamento assíncrono em todas as operações; o candidato deverá justificar onde ele agrega valor.

9. Uso de inteligência artificial

O uso de ferramentas de inteligência artificial é permitido e esperado. O objetivo do desafio não é avaliar exclusivamente a capacidade de escrever código manualmente, mas a capacidade de tomar decisões técnicas, validar resultados e compreender a solução construída.

O candidato deverá ser capaz de explicar a arquitetura, as principais regras de negócio, as decisões de modelagem e os testes implementados.

10. Entrega

O candidato deverá entregar:

- Código-fonte completo do projeto.
- Instruções claras para executar a aplicação localmente.
- Arquivos de configuração necessários, incluindo Docker.
- Testes automatizados, quando implementados.
- Histórico de troca de mensagens com a IA.

Não é necessário hospedar a aplicação em produção.